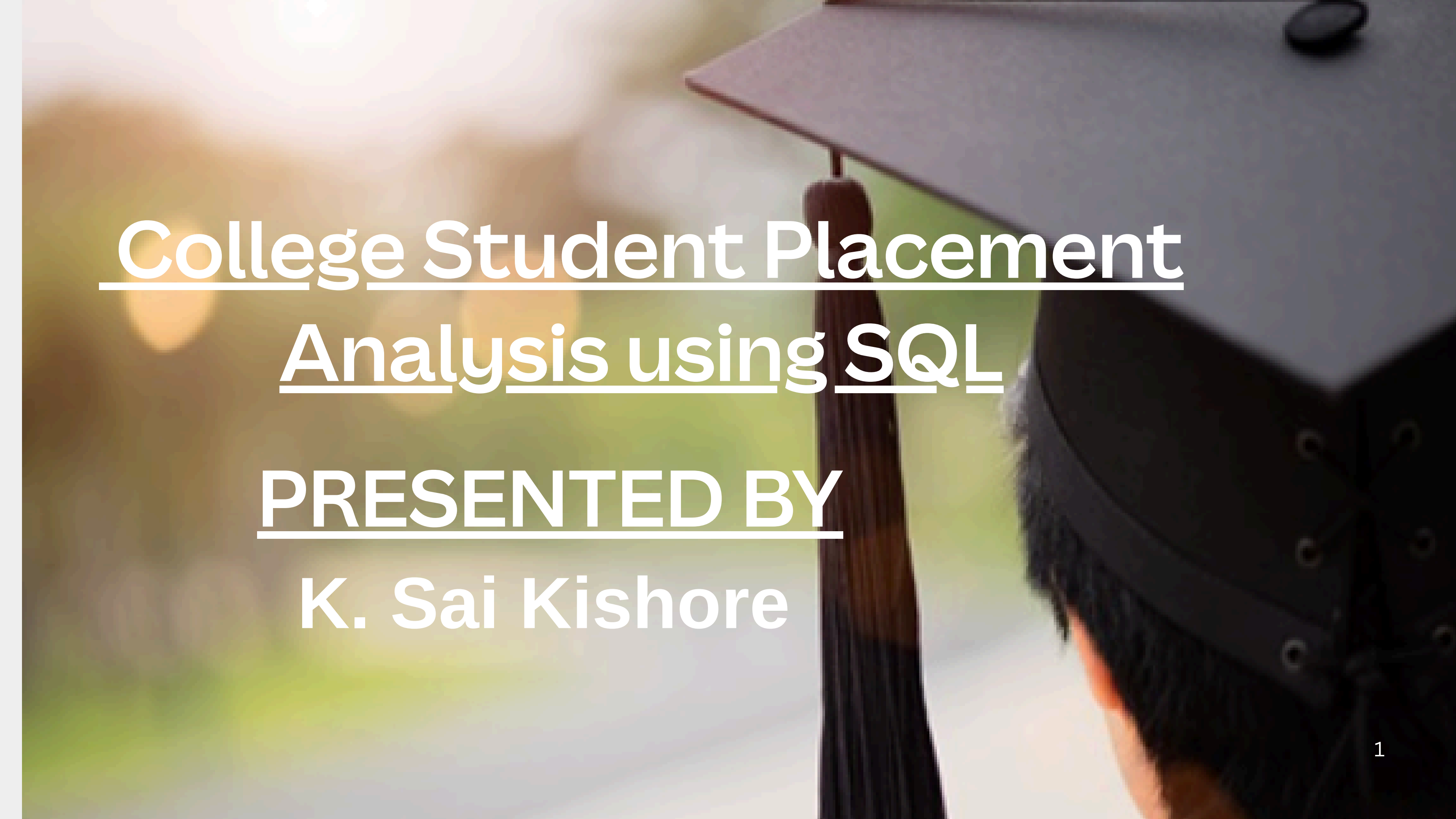




College Student Placement

Analysis using SQL

PRESENTED BY

K. Sai Kishore

INTRODUCTION

Placement of college students is a key measure of academic and institutional success. Analyzing placement data helps identify trends, improve training, and enhance employability. Factors studied include academic performance, skills, internships, and recruitment outcomes.

SQL is used to efficiently manage and analyze placement data from databases. Insights from SQL queries can help generate reports, dashboards, and improve placement strategies.

This project focuses on using SQL to analyze student placement and provide actionable insights.

ANALYSIS

- Overview:
- The database was designed to study the relationship between students' academic performance, experience, and placement outcomes.
- Data was collected from multiple tables:
- Students, AcademicPerformance, Experience, Skills, and Placement.
- Analysis queries were performed using SQL functions and conditions to identify patterns and insights.

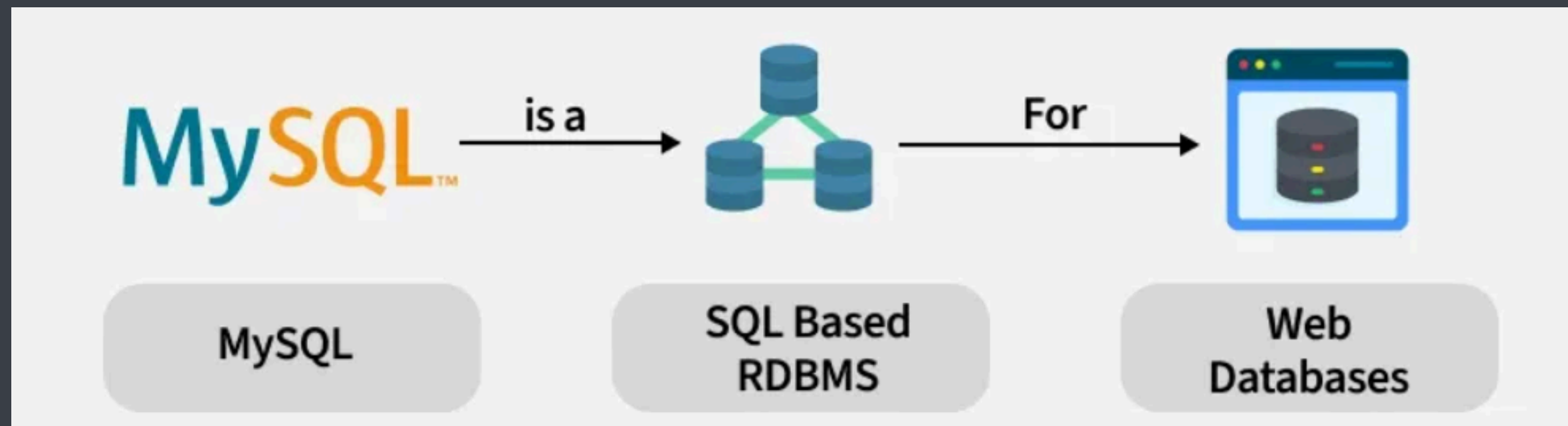
Analytical Findings:

- Average CGPA Analysis:
- Using AVG(CGPA), we found that students with a CGPA above 7.5 have a higher probability of placement.
- Placement Status Count:
- The COUNT() query revealed the number of students placed versus not placed, highlighting placement trends.
- IQ and Performance Correlation:
- Students with IQ above 100 often had better academic performance and higher CGPA values.
- Internship and Projects Impact:
- The Experience table showed that students with internship experience and more than 2 projects completed were more likely to be placed.
- Skill Development Observation:
- High communication skill scores and extra-curricular involvement improved overall placement chances.
- Top Performers:
- Queries using MAX() and ORDER BY helped identify top students in terms of CGPA and IQ.⁴

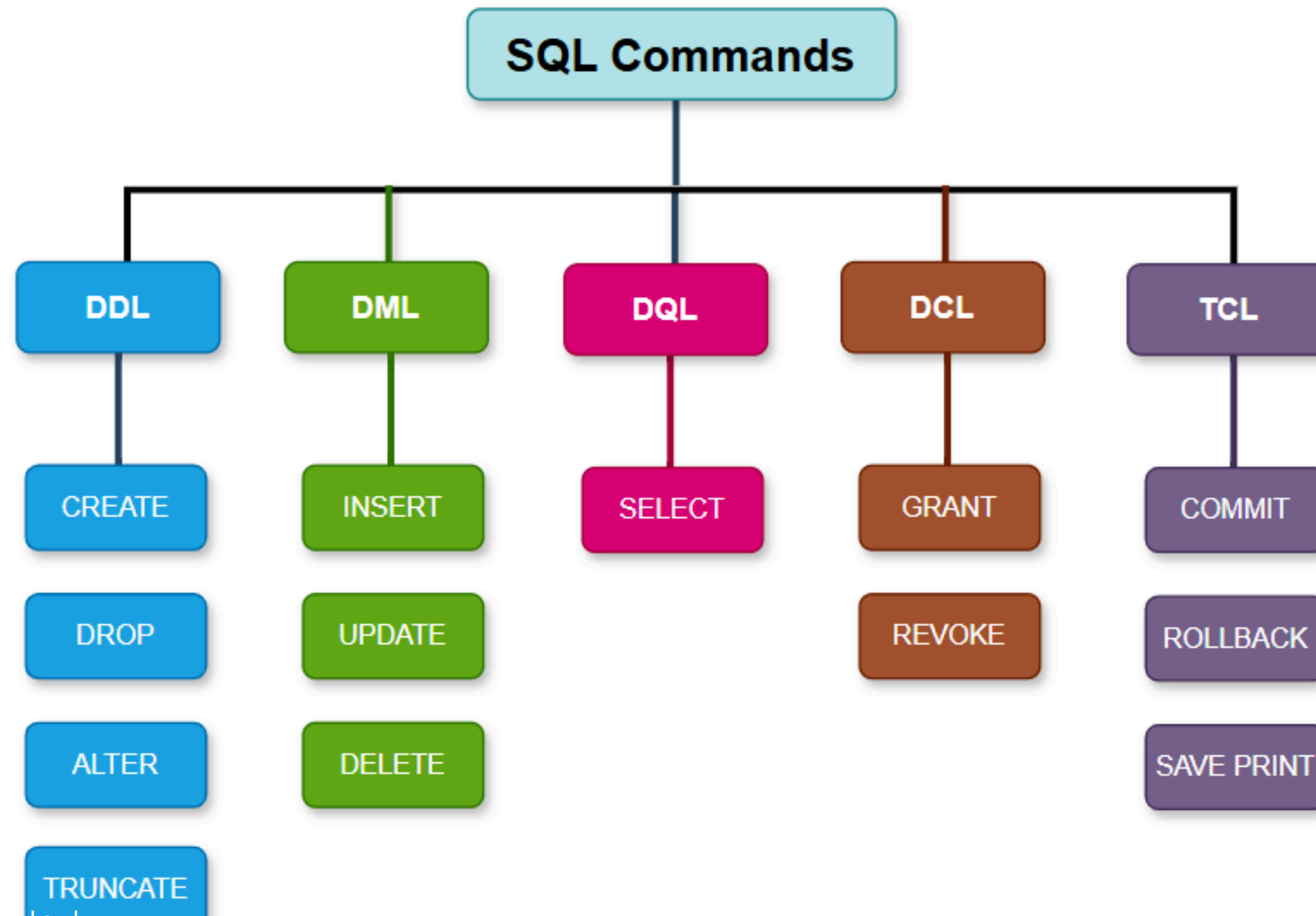
WHAT IS MYSQL?

MYSQL is an open_source relational database management system (RDBMS) that uses Structured Query Language (SQL) to store, organize , and retrieve data efficiently . It is one of the most widely used databases in the world, especially for web applications.

Relational Database - Data is stored in tables (rows and columns), and tables can be linked using relationships(primary keys & foreign keys).

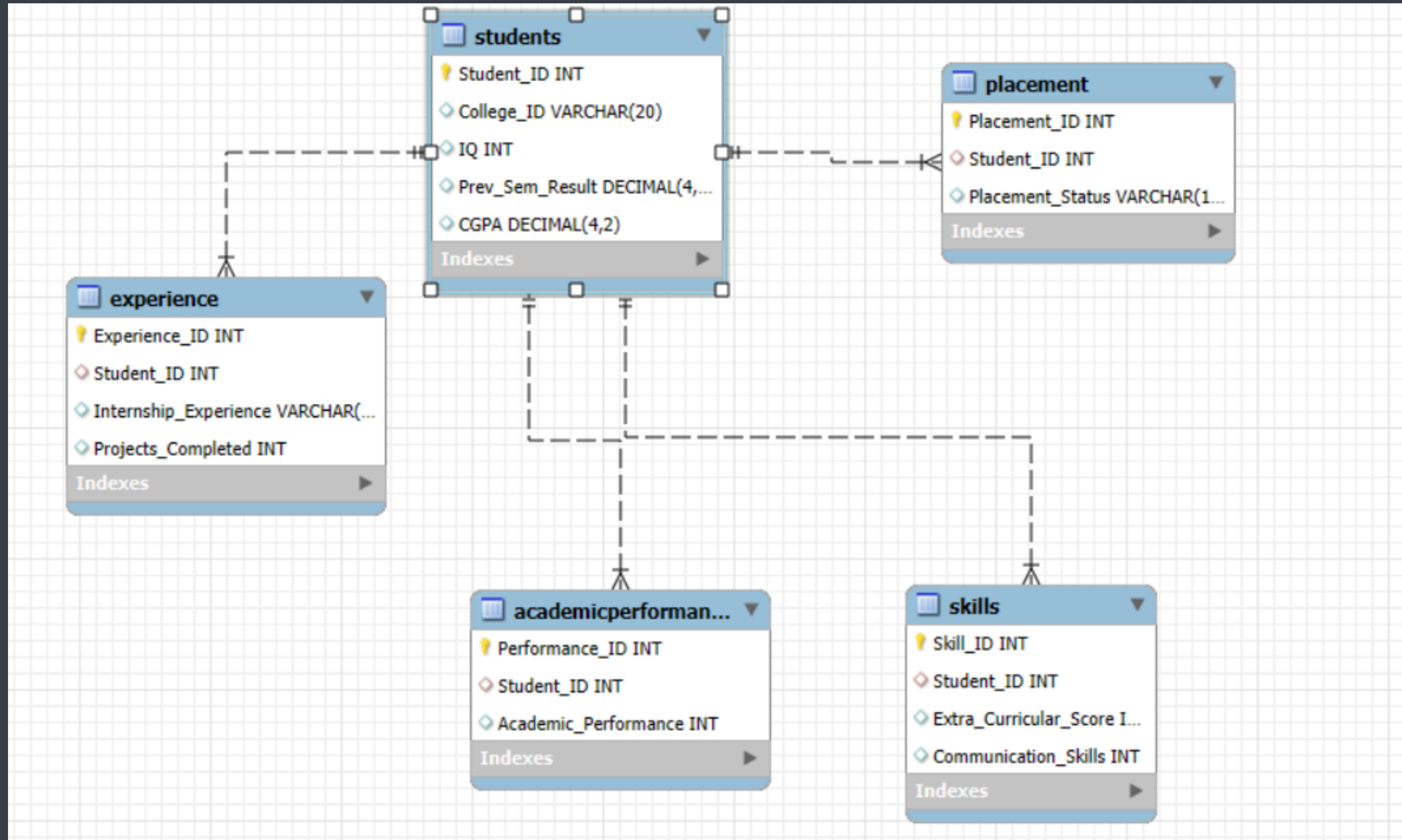


SQL COMMAND



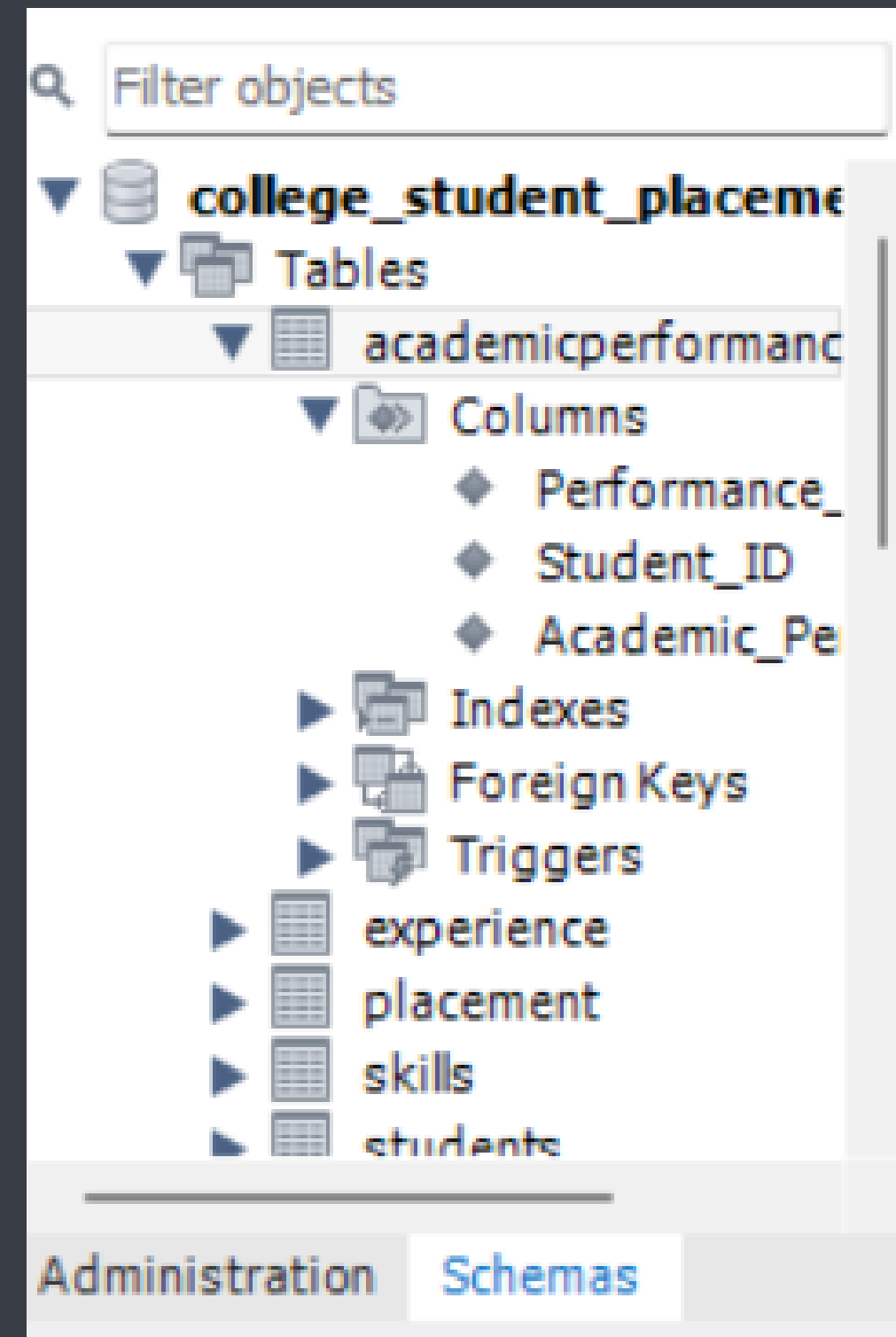
four paragraph text

ER DIAGRAM



CREATING DATABASE AND TABLES

- Create database College_Student_Placement
- Use College_Student_Placement



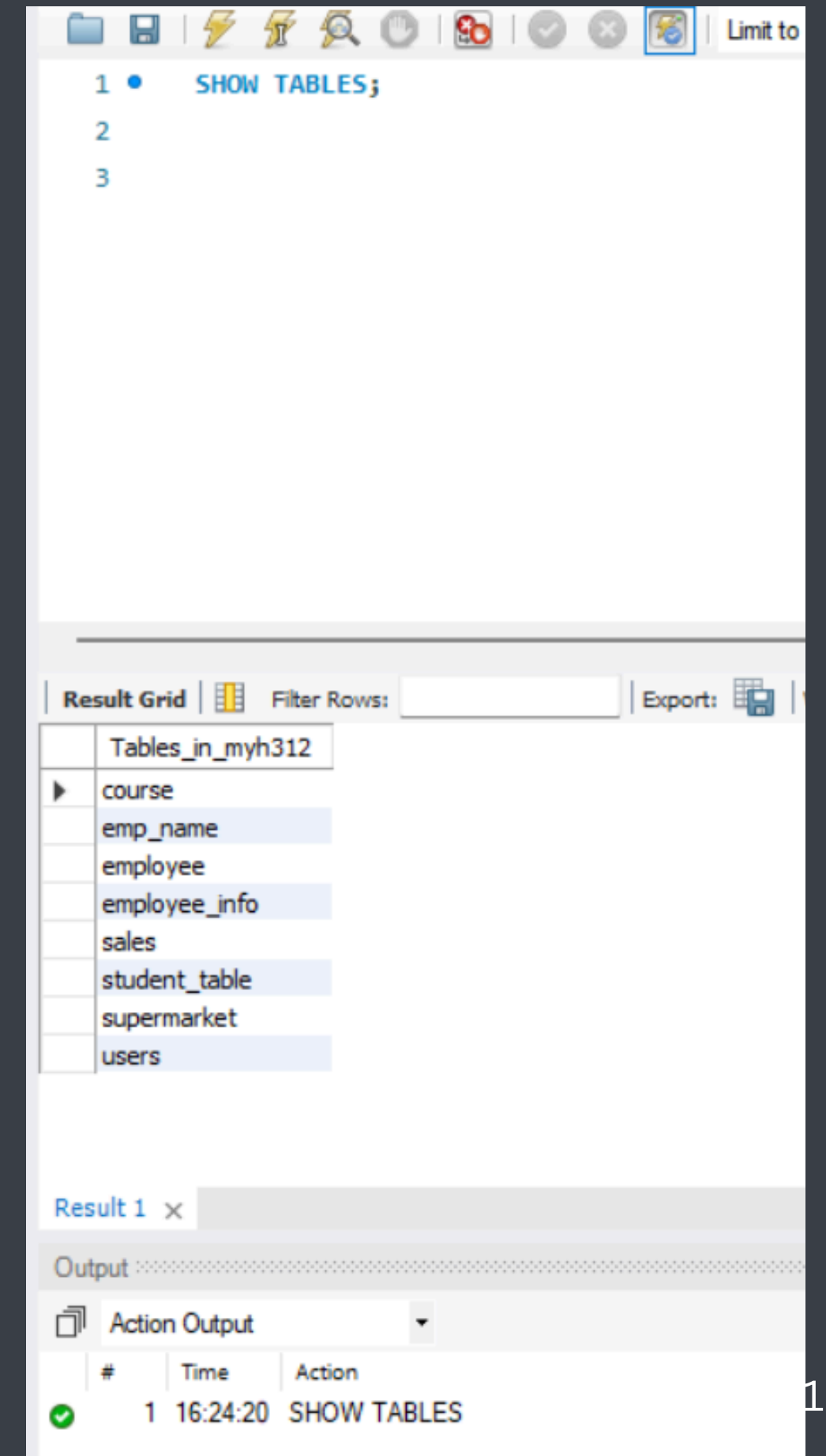
CREATING TABLES

```
CREATE TABLE Students(  
    Student_ID INT PRIMARY KEY AUTO_INCREMENT,  
    College_ID VARCHAR(20),  
    IQ INT,  
    Prev_Sem_Result DECIMAL(4,2),  
    CGPA DECIMAL(4,2)  
);  
  
SELECT * TABLE Students  
  
CREATE TABLE AcademicPerformance (  
    Performance_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Student_ID INT,  
    Academic_Performance INT,  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
);
```

```
CREATE TABLE Experience (  
    Experience_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Student_ID INT,  
    Internship_Experience VARCHAR(5), -- Yes/No  
    Projects_Completed INT,  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
);  
CREATE TABLE Skills (  
    Skill_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Student_ID INT,  
    Extra_Curricular_Score INT,  
    Communication_Skills INT,  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
);  
  
CREATE TABLE Placement (  
    Placement_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Student_ID INT,  
    Placement_Status VARCHAR(10), -- Yes/No  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
);
```

Verifying Database Tables using SHOW TABLES;

- After creating all the tables, it's important to verify whether they are successfully stored in the selected database.
- The SHOW TABLES; command in SQL lists all tables present in the current database.
- It helps confirm that your database structure is created correctly before inserting or querying data.
- This is a basic verification step before proceeding to data insertion or relationships.



Exploring Table Structures using DESCRIBE Command

- The DESC (or DESCRIBE) command is used to view the structure of a table in SQL.
- It helps in verifying that each table has the correct columns, data types, and constraints.
- This step is important before inserting or analyzing data.
- It displays information such as:
 - Field (Column name)
 - Type (Data type used)
 - Null (Whether NULL values are allowed)
 - Key (Primary or Foreign Key)
 - Default (Default value if any)
 - Extra (Auto Increment, etc.)

The screenshot shows a SQL IDE interface. The top pane contains a list of SQL commands: `DESCRIBE Students;`, `DESCRIBE AcademicPerformance;`, `DESCRIBE Experience;`, `DESCRIBE Skills;`, and `DESCRIBE Placement;`. The bottom pane displays the result grid for the `DESCRIBE Placement;` command. The result grid has columns: Field, Type, Null, Key, Default, and Extra. The data rows are:

Field	Type	Null	Key	Default	Extra
Placement_ID	int	NO	PRI	NULL	auto_increment
Student_ID	int	YES	MUL	NULL	
Placement_Status	varchar(10)	YES		NULL	

Below the result grid, there is an 'Output' section with a tab labeled 'Result 8'. The output shows a log of actions performed:

#	Time	Action
11	16:37:16	DESCRIBE AcademicPerformance
12	16:37:16	DESCRIBE Experience
13	16:37:16	DESCRIBE Skills
14	16:37:16	DESCRIBE Placement

INSERT THE DATA INTO EACH TABLE

-- Students Table

```
INSERT INTO Students (College_ID, IQ, Prev_Sem_Result, CGPA)
VALUES
('CLG0030', 107, 6.61, 6.28),
('CLG0061', 97, 5.52, 5.37),
('CLG0036', 109, 5.36, 5.83);
```

Then insert for the other tables:

-- AcademicPerformance

```
INSERT INTO AcademicPerformance (Student_ID, Academic_Performance)
VALUES (1, 85), (2, 70), (3, 78);
```

-- Experience

```
INSERT INTO Experience (Student_ID, Internship_Experience,
Projects_Completed)
VALUES (1, 'Yes', 2), (2, 'No', 1), (3, 'Yes', 3);
```

-- Skills

```
INSERT INTO Skills (Student_ID, Extra_Curricular_Score,
Communication_Skills)
VALUES (1, 80, 85), (2, 70, 60), (3, 90, 88);
```

-- Placement

```
INSERT INTO Placement (Student_ID, Placement_Status)
VALUES (1, 'Yes'), (2, 'No'), (3, 'Yes');
```


Verifying Inserted Data using SELECT Command

- After inserting data into the tables, we must verify whether the records were added successfully.
- The SELECT statement is used to retrieve and display data from one or more tables.
- It helps ensure that all columns contain correct and complete values.
- Commonly used after the INSERT INTO operation.
- You can display:
 - All columns → SELECT * FROM table_name;
 - Specific columns → SELECT column1, column2 FROM table_name;

The screenshot displays a database management interface. At the top, a list of SQL queries is shown, with the fifth query, `SELECT * FROM Placement;`, highlighted. Below the queries, a 'Result Grid' section shows the data retrieved from the 'Placement' table. The grid has three columns: 'Placement_ID', 'Student_ID', and 'Placement_Status'. The data rows are as follows:

Placement_ID	Student_ID	Placement_Status
1	1	No
2	2	No
3	3	No
4	1	Yes
5	2	No
6	3	Yes
NULL	NULL	NULL

Below the result grid, there are tabs for different tables: 'Students 1', 'AcademicPerformance 2', 'Experience 3', 'Skills 4', and 'Placement 5'. The 'Placement 5' tab is currently selected. At the bottom, an 'Action Output' section shows a log of executed queries and their execution times, all marked with green checkmarks to indicate success.

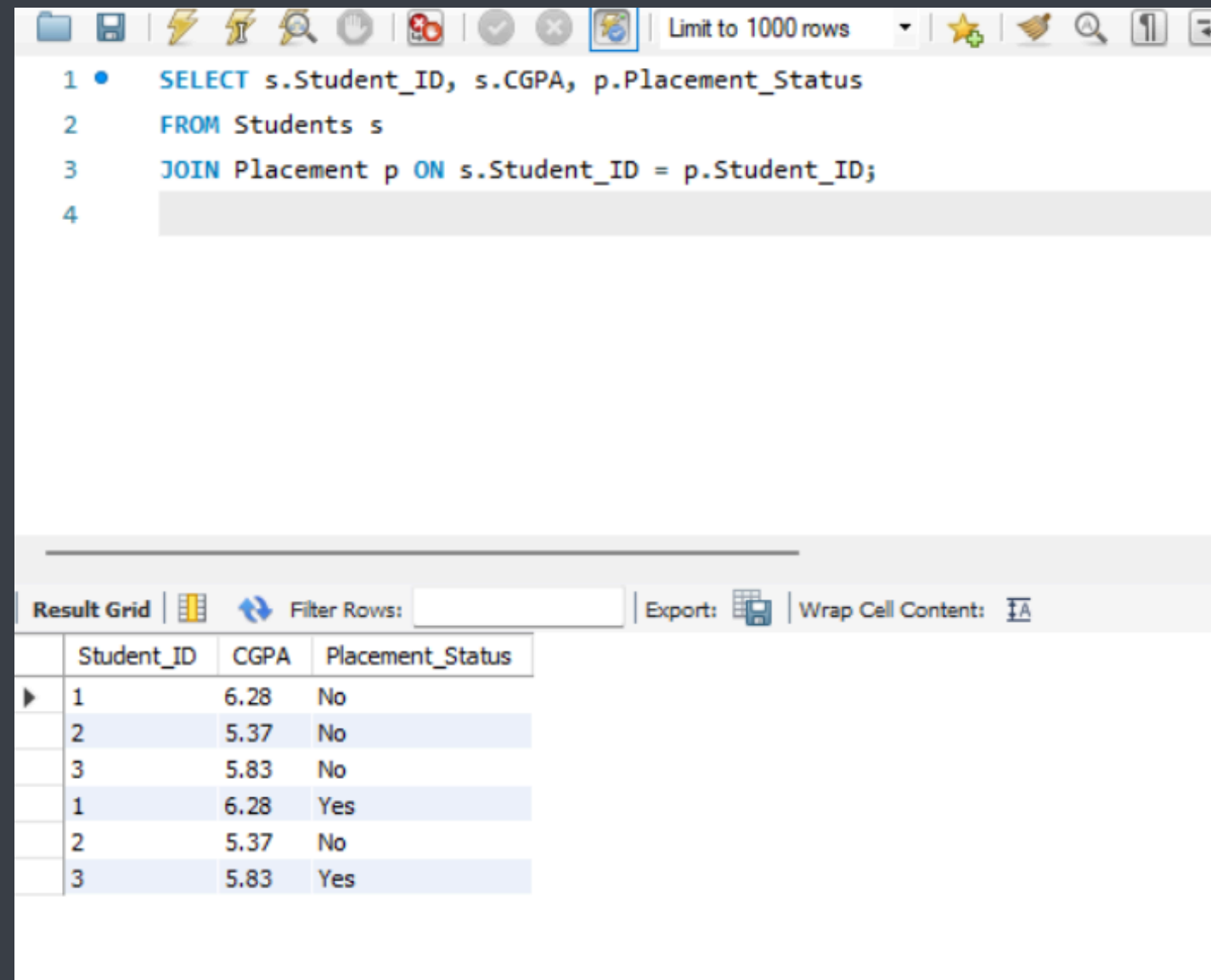
#	Time	Action
22	16:46:22	SELECT * FROM AcademicPerformance LIMIT 0, 1000
23	16:46:22	SELECT * FROM Experience LIMIT 0, 1000
24	16:46:22	SELECT * FROM Skills LIMIT 0, 1000
25	16:46:22	SELECT * FROM Placement LIMIT 0, 1000

Performing Analysis Queries in SQL

```
SELECT s.Student_ID, s.CGPA, p.Placement_Status  
FROM Students s
```

```
JOIN Placement p ON s.Student_ID = p.Student_ID;
```

```
SELECT s.Student_ID, e.Internship_Experience, p.Placement_Status
```



The screenshot shows a SQL query editor window with a toolbar at the top. The query is entered in a text area and is as follows:

```
1 • SELECT s.Student_ID, s.CGPA, p.Placement_Status  
2 FROM Students s  
3 JOIN Placement p ON s.Student_ID = p.Student_ID;  
4
```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid displays the following data:

	Student_ID	CGPA	Placement_Status
▶	1	6.28	No
	2	5.37	No
	3	5.83	No
	1	6.28	Yes
	2	5.37	No
	3	5.83	Yes


```
FROM Students s
JOIN Experience e ON s.Student_ID = e.Student_ID
JOIN Placement p ON s.Student_ID = p.Student_ID
WHERE e.Internship_Experience = 'Yes' AND p.Placement_Status = 'Yes';
SELECT AVG(CGPA) AS Average.CGPA FROM Students;
SELECT AVG(sk.Communication_Skills) AS Avg_Comm_Score
```

```
6 • SELECT s.Student_ID, e.Internship_Experience, p.Placement_Status
7 FROM Students s
8 JOIN Experience e ON s.Student_ID = e.Student_ID
9 JOIN Placement p ON s.Student_ID = p.Student_ID
10 WHERE e.Internship_Experience = 'Yes' AND p.Placement_Status = 'Yes';
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Student_ID	Internship_Experience	Placement_Status
1	Yes	Yes
3	Yes	Yes

```
SELECT AVG(CGPA) AS Average_CGPA FROM Students;
```

11

12 • `SELECT AVG(CGPA) AS Average_CGPA FROM Students;`

13

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Average_CGPA
▶	5.826667

```
SELECT AVG(sk.Communication_Skills) AS Avg_Comm_Score
      FROM Skills sk
JOIN Placement p ON sk.Student_ID = p.Student_ID
WHERE p.Placement_Status = 'Yes';
```

```
15 • SELECT AVG(sk.Communication_Skills) AS Avg_Comm_Score
16     FROM Skills sk
17     JOIN Placement p ON sk.Student_ID = p.Student_ID
18     WHERE p.Placement_Status = 'Yes';
19
20
21
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	Avg_Comm_Score
▶	60.3333

```
SELECT p.Placement_Status, AVG(a.Academic_Performance) AS Avg_Academic
      FROM Placement p
      JOIN AcademicPerformance a ON p.Student_ID = a.Student_ID
      GROUP BY p.Placement_Status;
```

```
20 • SELECT p.Placement_Status, AVG(a.Academic_Performance) AS Avg_Academic
21 FROM Placement p
22 JOIN AcademicPerformance a ON p.Student_ID = a.Student_ID
23 GROUP BY p.Placement_Status;
24
25
26
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	Placement_Status	Avg_Academic
▶	No	42.0000
	Yes	45.0000

OBSERVATIONS

- The database college_student_placement successfully stores all student-related information — including academic performance, experience, skills, and placement details.
- Using DESC and SELECT queries verified that table creation and data insertion were accurate and complete.
- Filtering queries using the WHERE clause helped identify high-performing students based on CGPA and IQ.
- Analytical queries such as AVG(), COUNT(), and MAX() provided meaningful insights into overall student performance and placement rates.
- Students with internship experience and strong communication skills showed a higher likelihood of being placed.
- The SQL database effectively demonstrates data relationships using foreign keys across multiple tables (Students, AcademicPerformance, Skills, Experience, and Placement).

CONCLUSION

- SQL provides a powerful and structured approach to manage, organize, and analyze college placement data.
- The project helps institutions understand the key factors influencing student placement outcomes.
- By using SQL queries and analysis, colleges can improve training programs, focus on skill development, and enhance placement success rates.
- This implementation showcases how data-driven decision-making can improve student performance and employability.

THANK YOU